

TRIE (PREFIX TREE)

MASTERCLASS

FAANG Discovery Journey — First Principles

📖 PHASE 1 — THE REAL-WORLD STORY

Every time you type a character into Google Search (e.g., `g` → `go` → `goo` → `goog`), Google instantly drops down word suggestions.

💡 THE DISCOVERY QUESTION

Google cannot scan millions of dictionary words on every single keystroke. How does it search prefixes in milliseconds?

😬 PHASE 2 — WHY HASHMAP FAILS FOR PREFIXES

- **HashMap Lookup:** Can find exact word "apple" in $O(L)$ time.
- **HashMap Prefix Search:** Fails! `contains("ap")` returns false. `contains("app")` returns false. Finding words with prefix "app" requires scanning ALL N dictionary keys → Slow $O(N \cdot L)$!

💡 PHASE 3 & AHA MOMENT

Can we reuse shared character paths?
Instead of storing "apple", "app", and "application" separately, store them along the shared path `a -> p -> p`!

Aha Moment: Every Character = Node | Whole Word = Path | Prefix = Shared Tree Path!

🔧 BUILD TRIE YOURSELF (VISUAL INSIGHT)

```
Root
  ↓
a -> p -> p (isWord=true)
      ↓
      l -> e (isWord=true)
      ↓
      i -> c -> a -> t -> i -> o
      -> n (isWord=true)
```

When inserting "application", the path `a -> p -> p` already exists! Only new characters are appended!

⚡ UNIVERSAL TRIE OPERATIONS

Operation	Algorithm Logic	Time
Insert(word)	Traverse <code>children[c - 'a']</code> . Create node if null. Set <code>isWord = true</code> at leaf.	$O(L)$
Search(word)	Traverse <code>children[c - 'a']</code> . Return <code>false</code> if null, else return <code>node.isWord</code> .	$O(L)$
StartsWith(prefix)	Traverse prefix. Return <code>true</code> as long as path exists (no <code>isWord</code> check needed!).	$O(L)$

TRIE (PREFIX TREE) MASTERCLASS

Java Master Toolkit & Utility Classes

★ THE BIGGEST AHA: WHY DFS + TRIE ARE COMBINED

Trie Controls: *VALID WORDS* (tells us if current prefix exists in dictionary).
DFS Controls: *VALID PATHS* (tells us where we can move on the 2D grid).

💡 Why Trie Stores `node.word` at Leaf?

Instead of building a `StringBuilder` on every recursive DFS call, store the full string `node.word` at terminal leaf nodes. When DFS reaches a leaf node, simply do `res.add(node.word)` in $O(1)$ time without string reconstruction!

PATTERN 1

IMPLEMENT TRIE (Prefix Tree Operations) e.g. Implement Trie (Prefix Tree) (LC 208)

WHEN TO USE

Implement `insert(word)`, `search(word)`, and `startsWith(prefix)` methods in $O(L)$ time.

TEMPLATE — LC 208

```
class Trie {
    static class Node {
        Node[] children = new Node[26];
        boolean isEnd = false;
    }
    private Node root = new Node();

    public void insert(String word) {
        Node node = root;
        for (char c : word.toCharArray()) {
            int idx = c - 'a';
            if (node.children[idx] == null) node.children[idx] = new Node();
            node = node.children[idx];
        }
        node.isEnd = true;
    }

    public boolean search(String word) {
        Node node = find(word);
        return node != null && node.isEnd;
    }

    public boolean startsWith(String prefix) {
        return find(prefix) != null;
    }

    private Node find(String str) {
        Node node = root;
        for (char c : str.toCharArray()) {
            int idx = c - 'a';
            if (node.children[idx] == null) return null;
            node = node.children[idx];
        }
        return node;
    }
}
```

PATTERN 2 WORD SEARCH II (Trie + 2D Grid DFS Pruning) e.g. Word Search II (LC 212)

GRID DFS + TRIE COMBO

Pass `TrieNode` directly in DFS. If `node.children[ch - 'a'] == null`, prune immediately! Set `node.word = null` after adding to result list to prevent duplicate additions.

TEMPLATE — LC 212

```
class Solution {
    static class TrieNode {
        TrieNode[] children = new TrieNode[26];
        String word = null;
    }

    public List<String> findWords(char[][] board, String[] words) {
        TrieNode root = new TrieNode();
        for (String w : words) {
            TrieNode node = root;
            for (char c : w.toCharArray()) {
                int idx = c - 'a';
                if (node.children[idx] == null) node.children[idx] = new TrieNode();
                node = node.children[idx];
            }
            node.word = w; // Store complete word at leaf node!
        }

        List<String> result = new ArrayList<>();
        for (int r = 0; r < board.length; r++) {
            for (int c = 0; c < board[0].length; c++) {
                dfs(board, r, c, root, result);
            }
        }
        return result;
    }

    private void dfs(char[][] board, int r, int c, TrieNode node, List<String> res) {
        char ch = board[r][c];
        if (ch == '#' || node.children[ch - 'a'] == null) return; // Prune branch!
        node = node.children[ch - 'a'];
        if (node.word != null) {
            res.add(node.word);
            node.word = null; // Prevent duplicate additions!
        }
        board[r][c] = '#'; // Mark visited!
        int[] dr = {-1, 1, 0, 0}, dc = {0, 0, -1, 1};
        for (int i = 0; i < 4; i++) {
            int nr = r + dr[i], nc = c + dc[i];
            if (nr >= 0 && nr < board.length && nc >= 0 && nc < board[0].length) {
                dfs(board, nr, nc, node, res);
            }
        }
        board[r][c] = ch; // Backtrack!
    }
}
```

PATTERN
3

DESIGN ADD AND SEARCH WORDS (Wildcard Search)

e.g. Design Add and Search Words Data Structure (LC 211)

WILDCARD DFS SEARCH

If char is '.', branch to all non-null children nodes. Otherwise follow exact character branch index `c - 'a'`.

TEMPLATE — LC 211

```
class WordDictionary {
    static class Node {
        Node[] children = new Node[26];
        boolean isEnd = false;
    }
    private Node root = new Node();

    public void addWord(String word) {
        Node node = root;
        for (char c : word.toCharArray()) {
            int idx = c - 'a';
            if (node.children[idx] == null) node.children[idx] = new Node();
            node = node.children[idx];
        }
        node.isEnd = true;
    }

    public boolean search(String word) {
        return dfs(word, 0, root);
    }

    private boolean dfs(String word, int idx, Node node) {
        if (node == null) return false;
        if (idx == word.length()) return node.isEnd;
        char c = word.charAt(idx);
        if (c != '.') {
            return dfs(word, idx + 1, node.children[c - 'a']);
        }
        for (int i = 0; i < 26; i++) {
            if (node.children[i] != null && dfs(word, idx + 1, node.children[i])) return true;
        }
        return false;
    }
}
```

PATTERN 4

REPLACE WORDS (Shortest Dictionary Root Match) e.g. Replace Words (LC 648)

SHORTEST ROOT REPLACEMENT

Insert roots into Trie. For each word in sentence, traverse Trie. Stop and replace as soon as `node.isEnd == true`!

TEMPLATE — LC 648

```
public String replaceWords(List<String> dictionary, String sentence) {
    Trie trie = new Trie();
    for (String root : dictionary) trie.insert(root);
    String[] words = sentence.split(" ");
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < words.length; i++) {
        if (i > 0) sb.append(" ");
        sb.append(findRoot(words[i], trie.root));
    }
    return sb.toString();
}

private String findRoot(String word, TrieNode root) {
    TrieNode node = root;
    StringBuilder prefix = new StringBuilder();
    for (char c : word.toCharArray()) {
        int idx = c - 'a';
        if (node.children[idx] == null || node.isEnd) break;
        prefix.append(c);
        node = node.children[idx];
    }
    return node.isEnd ? prefix.toString() : word;
}
```

TRIE (PREFIX TREE)

MASTERCLASS

Decision Tree & Trigger Words

COMPLETE TRIE PROBLEM LADDER SUMMARY					
LC #	Problem	Pattern	Time	Space	Diff
208	Implement Trie	Standard Trie Operations	$O(L)$	$O(\text{Total Chars})$	M
211	Design Add & Search Words	Wildcard Dot DFS	$O(26^L)$ worst	$O(L)$ stack	M
648	Replace Words	Shortest Root Match	$O(L)$ per word	$O(\text{Total Chars})$	M
212	Word Search II	Trie + 2D Grid DFS Pruning	$O(M \cdot N \cdot 4^L)$	$O(\text{Total Chars})$	H

TRIE (PREFIX TREE)

MASTERCLASS

Curated Problem Ladder — Part 1

LC 208 · Implement Trie

MEDIUM

Pattern: Standard Trie

Key Insight: `children[26]` array.

```
public void insert(String word) {
    Node node = root;
    for (char c : word.toCharArray())
    {
        int idx = c - 'a';
        if (node.children[idx] == null)
            node.children[idx] = new Node();
        node = node.children[idx];
    }
    node.isEnd = true;
}
```

LC 211 · Design Add & Search Words

MEDIUM

Pattern: Wildcard DFS

Key Insight: Branch 26 children on '.'.

```
private boolean dfs(String w, int i, Node n) {
    if (n == null) return false;
    if (i == w.length()) return n.isEnd;
    char c = w.charAt(i);
    if (c != '.') return dfs(w, i + 1, n.children[c - 'a']);
    for (int j = 0; j < 26; j++)
        if (n.children[j] != null && dfs(w, i + 1, n.children[j])) return true;
    return false;
}
```

LC 648 · Replace Words

MEDIUM

Pattern: Shortest Root Match

Key Insight: Stop on first `isEnd == true`.

```
private String findRoot(String w, Node root) {
    Node n = root;
    StringBuilder prefix = new StringBuilder();
    for (char c : w.toCharArray()) {
        int idx = c - 'a';
        if (n.children[idx] == null || n.isEnd) break;
        prefix.append(c);
        n = n.children[idx];
    }
    return n.isEnd ? prefix.toString() : w;
}
```


TRIE (PREFIX TREE)

MASTERCLASS

Curated Problem Ladder — Part 2

LC 212 · Word Search II		HARD
Pattern: Grid DFS + Trie Combo Key Insight: Pass `TrieNode` in grid DFS, nullify `node.word`.	<pre>private void dfs(char[][] b, int r, int c, Node n, List<String> res) { char ch = b[r][c]; if (ch == '#' n.children[ch - 'a'] == null) return; n = n.children[ch - 'a']; if (n.word != null) { res.add(n.word); n.word = null; } b[r][c] = '#'; int[] dr = {-1,1,0,0}, dc = {0,0,-1,1}; for (int i = 0; i < 4; i++) { int nr = r + dr[i], nc = c + dc[i]; if (nr >= 0 && nr < b.length && nc >= 0 && nc < b[0].length) dfs(b, nr, nc, n, res); } }</pre>	

TRIE (PREFIX TREE)

MASTERCLASS

Dry Run, Proofs & Mastery Cheat Sheet

🔍 DRY RUN — Word Search II Trie Pruning

Grid Pos	Char	Trie Child ('n.children[ch - 'a'])	Action / Result
(0, 0)	'a'	Non-null (points to 'a' node)	Continue DFS to neighbours
(0, 1)	'p'	Non-null (points to 'p' node)	Continue DFS to neighbours
(0, 2)	'z'	NULL!	PRUNE IMMEDIATELY! Backtrack! ✅

📐 MATHEMATICAL PROOF — $O(L)$ INDEPENDENCE FROM DICTIONARY SIZE N

Claim: Searching a prefix of length L in a Trie takes $O(L)$ time regardless of whether the dictionary contains 100 or 10,000,000 words.

Proof: At each character step i , the algorithm performs 1 array index lookup `children[c - 'a']` in $O(1)$ time. For prefix of length L , exactly L steps are taken $\implies \mathbf{O(L)}$ Time, Independent of N ! $\$$

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

☐ Implement Trie (`insert`, `search`, `startsWith`) in 5m

☐ Design TrieNode with `Node[26]` and `String word`

☐ Write Wildcard Search with `.'` recursion

☐ Code Replace Words with shortest root match

☐ Implement Word Search II (LC 212) Trie + Grid DFS

☐ Prune grid DFS when `n.children[ch - 'a'] == null`

☐ Nullify `node.word = null` after match to prevent duplicates

☐ Prove why Trie search runtime is independent of N

📦 GOLDEN RULES — NEVER FORGET

Rule 1 — Store Complete Word String at Leaf Nodes for Grid DFS
In Word Search II (LC 212), ALWAYS store `node.word = word` inside the Trie leaf node! This avoids reconstructing the string during 2D grid DFS and speeds up execution by 5x!

Rule 2 — Nullify Word After Result Addition
When a word match is found during grid DFS, set `node.word = null` immediately after adding to `result` list. This prevents adding duplicate instances of the same word found via different grid paths!